

Meeting 18/03/2026

Summary

Action Items & Next Steps

- LocalStack: enforce IAM roles and policies; all Boto3 operations must authenticate via IAM - no open access
- Add structured logging (logger), try/except/finally blocks, and argparse-based CLI entrypoints to all Python scripts interacting with LocalStack
- Implement S3 Intelligent-Tiering: detect whether it is enabled and configure it via Boto3; bundle the full LocalStack + REST API interaction into a Docker container
- Produce OpenAPI specifications for all REST APIs (S3, DynamoDB operations); publish spec to the shared GitHub documentation repository
- Set up API documentation portal: evaluate Redocly and MkDocs; render OpenAPI specs and host developer docs with versioned releases
- Define the doc repo branching model: local branch → dev branch (draft docs) → master (published docs available to all consumers)
- LDAP Server: host via Docker image; integrate with Jenkins over HTTPS with SSL (self-signed certificate); configure TLS termination
- Configure Nginx as an HTTP reverse proxy in front of the artifact repository; ensure persistent volume for Nexus data; test upgrade/downgrade paths
- Helm chart creation for all microservices; deploy to 2-node Kubernetes cluster with Istio VirtualService and DestinationRule
- Jaeger tracing: produce a shared OpenAPI spec for the college frontend; instrument services with OpenTelemetry and verify traces end-to-end in Jaeger
- Loki alerting: write LogQL expressions to grep 5xx errors from ingested logs and trigger notifications (email, SMS, Slack, Teams)
- Prometheus alerting: define SLA-based alerting rules on time-series metrics; route alerts to email, SMS, push notification, Slack, and Teams via Alertmanager
- Tech discussion required: finalise database design — DynamoDB as primary store, evaluate need for a vector DB alongside it, and confirm SNS + CloudWatch Metrics on LocalStack
- Review what automated checks should run on the doc repo on PR (lint, broken links, schema validation against OpenAPI spec)

Work Reviewed & Concepts Discussed

LocalStack, IAM & Python Best Practices

- LocalStack now requires IAM enforcement: all AWS SDK calls must use IAM roles and policies rather than open credentials — mirrors real AWS security posture
 - S3 Intelligent-Tiering: an AWS storage class that automatically moves objects between access tiers based on usage patterns; team to detect current tiering status and configure it programmatically via Boto3
 - Python code standards enforced going forward: every script must use the standard logging module (not print), wrap all AWS calls in try/except with specific exception types
-

- Bundling approach: the LocalStack interaction layer (REST API + Boto3 operations) is to be containerised — a Docker image that exposes HTTP endpoints for S3 and DynamoDB operations, runnable standalone or as a compose service

API Documentation & Developer Portal

- OpenAPI Specification (OAS 3.x): all REST APIs must have a machine - readable spec file before they are considered published; spec covers paths, request/response schemas, authentication, and error codes
- Redocly: a documentation portal and OpenAPI linting tool; renders specs into a clean, navigable reference site with try-it-out support evaluated as the primary docs renderer.
- MkDocs: static site generator for markdown-based developer guides; complements OpenAPI specs with conceptual documentation, architecture diagrams, and tutorials to be hosted via GitHub Pages or a container
- GitHub documentation repository structure: a dedicated repo holds all specs and docs; branching model mirrors the code workflow:
 - Local/feature branch: API under development, spec is a draft
 - Dev branch: spec reviewed and accurate for the current dev build developers read docs [here](#)
 - Master branch: published spec, versioned (e.g. v1.2.3) this is what external consumers and the college lab environment see
- Versioning example: an API developed with 10 endpoints gets its spec merged to master at v1.2.3; the college lab pulls from master and sees only the published, stable surface
- Doc repo checks on PR: automated gates should include OpenAPI schema lint (Redocly CLI), broken internal link detection, and markdown formatting checks PR is blocked if any gate fails

LDAP, SSL & Jenkins Integration

- LDAP Server hosted as a Docker container (OpenLDAP image); acts as the central identity provider for both Jenkins and Nexus single source of truth for user accounts and group membership
- Jenkins LDAP integration: Jenkins configured to authenticate against the LDAP server; groups in LDAP map to Jenkins roles (admin, developer, read-only)
- HTTPS / SSL for Jenkins: self-signed certificate generated and installed; Jenkins reverse-proxied behind Nginx with TLS termination all traffic to Jenkins is encrypted in transit
- SSL certificate concepts covered: certificate signing requests (CSR), self-signed vs CA-signed certificates, certificate chain, and how browsers and tools validate them

Nginx Reverse Proxy & Artifact Repository

- Nginx configured as an HTTP reverse proxy in front of Sonatype Nexus: all artifact publish/pull requests go through Nginx, which handles TLS termination and forwards to Nexus on the internal network
- Docker image tagging convention established: images are tagged with the repository path and version before pushing to Nexus (e.g. localhost:9082/jcrec/jcrec_docs:2.1) — tag format encodes registry host, namespace, repository name, and semantic version

- Persistent volumes for Nexus: Nexus stores artifact metadata and blobs on disk; a named Docker volume (or Kubernetes PersistentVolumeClaim) is required so data survives container restarts and upgrades
- Upgrade/downgrade paths: because Nexus data is externalised to a volume, upgrading the Nexus container image does not lose stored artifacts downgrade is possible by reverting the image tag while keeping the volume intact

Helm & Kubernetes — 2-Node Cluster with Istio

- Helm chart creation: each microservice packaged as a Helm chart with configurable values.yaml enables environment-specific overrides (dev vs prod) without changing chart templates
- Sonatype Nexus as Helm repository: charts pushed to Nexus after packaging (helm package) and served over HTTP Jenkins pipeline publishes a new chart version on every successful build
- 2-node Kubernetes cluster set up using VMs (kubeadm): one control-plane node and one worker node; demonstrated that workloads schedule onto the worker while the control plane manages cluster state
- Istio on the 2-node cluster: VirtualService and DestinationRule configured for traffic splitting allows weighted routing between service versions (foundation for canary deployments later)

Observability - Loki, Prometheus & Jaeger

- Loki log mining: LogQL expressions written to filter ingested logs. Example: grep all 5xx HTTP errors from a specific service label and count them over a time window
- Loki-based alerting: a LogQL alert rule fires when the 5xx error rate crosses a threshold; Grafana Alertmanager routes the notification to email, SMS, Slack, and Teams
- Prometheus SLA monitoring: time-series metrics (request latency p99, error rate, uptime) compared against SLA thresholds defined as recording rules; when an SLA is breached an alert fires via Alertmanager to the same notification channels
- Prometheus Alertmanager notification channels: email, SMS, PagerDuty, push notification, Slack, and Teams all configured as receivers with routing rules based on alert severity and label
- Jaeger tracing work continued: team to produce a shared OpenAPI spec for the college frontend API so Jaeger can correlate traces with named operations; OpenTelemetry instrumentation to be added to React and FastAPI
- OAuth2: a free/open-source OAuth2 server reviewed as the auth layer for the API will issue tokens consumed by the frontend and validated by the backend

Database Design & Technology Discussion

- Primary database: DynamoDB (via LocalStack) confirmed as the main data store single-table design, partition key strategy, and GSI usage to be finalised in a dedicated tech discussion
- Vector database consideration: raised as an open question, if semantic search or AI-assisted features are added (e.g. similarity search on alumni profiles), a vector DB (pgvector, Weaviate, or Pinecone) may be needed alongside DynamoDB
- SNS on LocalStack: to be configured for pub/sub notification flows services publish events to SNS topics, Lambda or SQS consumers subscribe and process them

